National College of Computer Studies

Tribhuvan University

Bsc. CSIT

Sixth Semester

Report on: FIFA Squad Builder

Submitted By:
Name: Ayush Tuladhar
Roll No: 7
Section: A

Submitted To:
Yuba Raj Devkota
Department of Computer Science and Information Technology
National College of Computer Studies
Paknajol Marg, Kathmandu

…………………………..
Signature

Acknowledgement

I would like to express my sincere gratitude to my teacher for their guidance during the preparation of this project report. Their feedback helped me in understanding the requirements, structure, and documentation standards needed for an organized academic project report.

I am also thankful to everyone who supported the development and testing of this FIFA Squad Builder application. The project helped me apply concepts of relational database design, secure authentication and authorization, RESTful server-side architecture, algorithm design, and software engineering practices in a practical ASP.NET Core MVC and C# based full-stack development environment.

Abstract

FIFA Squad Builder is a full-stack web application developed using ASP.NET Core MVC and C#, backed by a MySQL database populated from a real FIFA 23 player dataset. The application allows registered users to build football squads by selecting real players onto a formation-driven pitch, with live statistics - squad rating, chemistry, average age and potential, squad value, and wage-budget status - updating as the squad is built. An administrator role manages the underlying player and formation data, imports the dataset, and manages user accounts. This documentation presents the analysis, design, and implementation of the system. It describes the problem statement, objectives, feasibility, system design, module-wise implementation, and result analysis of the project. The report includes the Use Case Diagram as the required visual model for understanding user interaction with the system.

.

Table of Contents

1. Introduction

FIFA Squad Builder is a full-stack web application developed as an academic project using the ASP.NET Core MVC framework and the C# programming language. The application allows a registered user to build a football squad the way EA's Ultimate Team mode does: pick real players from an imported FIFA 23 dataset, arrange them on a chosen formation, and see squad rating, chemistry, average age and potential, squad value, and wage-budget status update live as the squad changes. An administrator role manages the underlying player and formation data, runs the dataset import, and manages other users' access.

1.1. Purpose

The purpose of this project is to design and develop a secure, role-based, full-stack web application that demonstrates practical application of relational database design, server-side business-rule enforcement, RESTful MVC architecture, and original algorithm design, within a realistic scope for a semester-length Bachelor's project.

1.2. Project Scope

The scope of the project includes user registration and login, two roles (User and Admin), creation and management of multiple squads per user, a formation-driven pitch editor with server-side player search and pagination, duplicate-player prevention and squad-ownership enforcement at the database level, an original chemistry/rating/weak-position calculation system, admin CRUD for players and formations, an idempotent CSV import pipeline for the real FIFA 23 dataset, and an admin screen for managing user roles. Drag-and-drop player placement and background audio/animation effects were intentionally excluded from scope in favor of a simpler, more reliable click-to-search assignment flow.

1.3. Document Convention

This document follows a standard academic project report format. The main text uses numbered headings and sub-headings, with sample code shown in a shaded monospace block and figures captioned below each image. Screenshots and diagrams are marked as placeholders in this version of the report, to be replaced with the author's own captured images.

1.4. Additional Information

- Framework: ASP.NET Core MVC (.NET 8 LTS)
- Programming Language: C#
- Database: MySQL 8, accessed via Entity Framework Core 8 and the Pomelo.EntityFrameworkCore.MySql provider
- Authentication: ASP.NET Core Identity (role-based: User, Admin)
- IDE: Visual Studio

- Frontend: Razor Views, vanilla JavaScript, hand-written CSS (with Bootstrap utility classes for base layout)
- Data Import: CsvHelper, against the real FIFA 23 complete player dataset (players_fifa23.csv, teams_fifa23.csv)
- Testing: xUnit with the Entity Framework Core InMemory provider

2. Problem Statement

Squad-building and "best XI" discussions are one of the most common ways football fans engage with player data, and EA's FIFA Ultimate Team mode is the best-known example of turning that into an interactive product. However, that product is closed - it runs inside a commercial game, uses undocumented proprietary rating and chemistry formulas, and gives a student or hobbyist no way to inspect, modify, or learn from how it actually works.

At the same time, real FIFA player data is freely available as flat CSV files but turning eighteen and a half thousand unrelated player and club records into something a person can actually browse, filter, and build a squad from requires genuine relational database design, server-side business logic, and a real user interface - none of which a spreadsheet provides.

Therefore, this project aims to develop FIFA Squad Builder, a web application using ASP.NET Core MVC and C# that imports a real, messy, undocumented-schema dataset into a normalized database, exposes it through server-side search and pagination, enforces genuine ownership and business rules for each user's squads, and implements its own transparent, documented chemistry, rating, and weak-position algorithms rather than claiming to reproduce any proprietary formula.

3. Objectives

The objectives of this project are:

- To design and develop a normalized, real-data-backed relational database, importing a genuine FIFA 23 player dataset into MySQL via a reusable, idempotent import service.
- To implement a secure, role-based, multi-user web application - with server-enforced squad ownership, duplicate-player prevention, and admin-only management of the master player and formation data - using ASP.NET Core Identity, MVC, and Entity Framework Core.
- To design and implement original, testable calculation services - chemistry, squad rating, and weak-position analysis - as isolated, unit-tested services rather than logic embedded in views or controllers.

4. Feasibility Study

4.1. Technical Feasibility

The project uses ASP.NET Core, a free and widely documented framework, together with C#, a language commonly taught within Computer Science and IT curricula. All required systems - Entity Framework Core, ASP.NET Core Identity, and the MVC pattern - are built into or officially supported by the framework itself, and a free, publicly available FIFA 23 player dataset was used for the underlying data. No paid software, proprietary server infrastructure, or specialized hardware was required, which made the project technically feasible to complete on a personal computer with a local MySQL installation.

4.2. Schedule Feasibility

The project was broken down into small, sequential development phases so that each system (database schema, authentication, data import, squad management, calculation services, UI) was built, tested, and confirmed working before moving to the next. This phase-based approach kept the workload manageable within the project timeline. The major development phases are summarized below.

Phase	Focus Area
1	Requirements analysis, database schema design, and algorithm definitions
2	Project scaffolding: MVC setup, Entity Framework Core, Identity, and MySQL configuration
3	Domain entities, relationships, and formation/position seed data
4	CSV import service and admin CRUD for players and formations
5	Authentication, role-based authorization, and admin account bootstrap
6	Squad creation, player assignment, and duplicate-prevention rules
7	Chemistry, squad-rating, and weak-position calculation services with unit tests
8	Squad pitch UI, player-card styling, and the player-search picker
9	Admin UI for players, formations, and user-role management
10	Testing, bug fixing, and documentation

4.3. Operational Feasibility

The finished application runs as a standard browser-based web application backed by a local or network-reachable MySQL server. It does not depend on any third-party paid service, and the admin tooling (dataset import, player/formation management, user management) is reachable through the same web interface rather than a separate command-line-only process, making it straightforward to operate and demonstrate.

5. System Analysis and Design

5.1. System Overview:

The system is a multi-user, browser-based web application. A user interacts with the system entirely through a web browser, registering an account, creating one or more squads, and assigning real players to a chosen formation. Every squad-related action is validated server-side against the requesting user's identity, so a user can only ever view or modify their own squads. An Administrator account manages the master Player and Formation data, runs the dataset import, and manages other users' roles.

5.2. Major Modules

- Authentication & Authorization Module - registration, login, logout, and role-based access control
- Player Data Import Module - idempotent CSV import of the real FIFA 23 dataset into MySQL
- Admin Management Module - CRUD for players and formations, and user role management
- Squad Management Module - create, rename, delete, and list a user's own squads
- Player Assignment Module - formation-driven pitch, player search, and slot/bench assignment
- Calculation Module - chemistry, squad rating, average age/potential, and squad value/wages
- Weak-Position Module - compares each starter against the best available unused alternative

5.3. User Class

There are two user classes for this system:

- User - a registered account that can create, edit, and delete its own squads, search and assign real players, switch formations, and view squad statistics. A user can never access or modify another user's squads.
- Admin - has all the abilities of a User, plus full CRUD over the master Player and Formation data, dataset import, and the ability to promote or demote other users' Admin status. Admin status can never be granted through self-registration - it is only ever assigned by an existing Admin or a configuration-driven startup bootstrap.

5.4. Operating Environment

The application runs as an ASP.NET Core web server process, reachable through any standard web browser, backed by a MySQL 8 database. It requires the .NET 8 runtime and a reachable

MySQL instance, but no client-side installation beyond a browser - all administration (including the dataset import) is performed through the same web interface.

5.5. Use Case Diagram

The primary actors of the system are the User and the Admin. The main use cases include: Register Account, Login/Logout, Create Squad, Search & Filter Players, Assign Player to Slot/Bench, Change Formation, View Squad Statistics & Weak Positions, Delete Squad, Manage Players (CRUD), Manage Formations & Pitch Slots, Import Player Dataset, and Manage User Roles. The figure below shows the relationship between the two actors and these use cases.

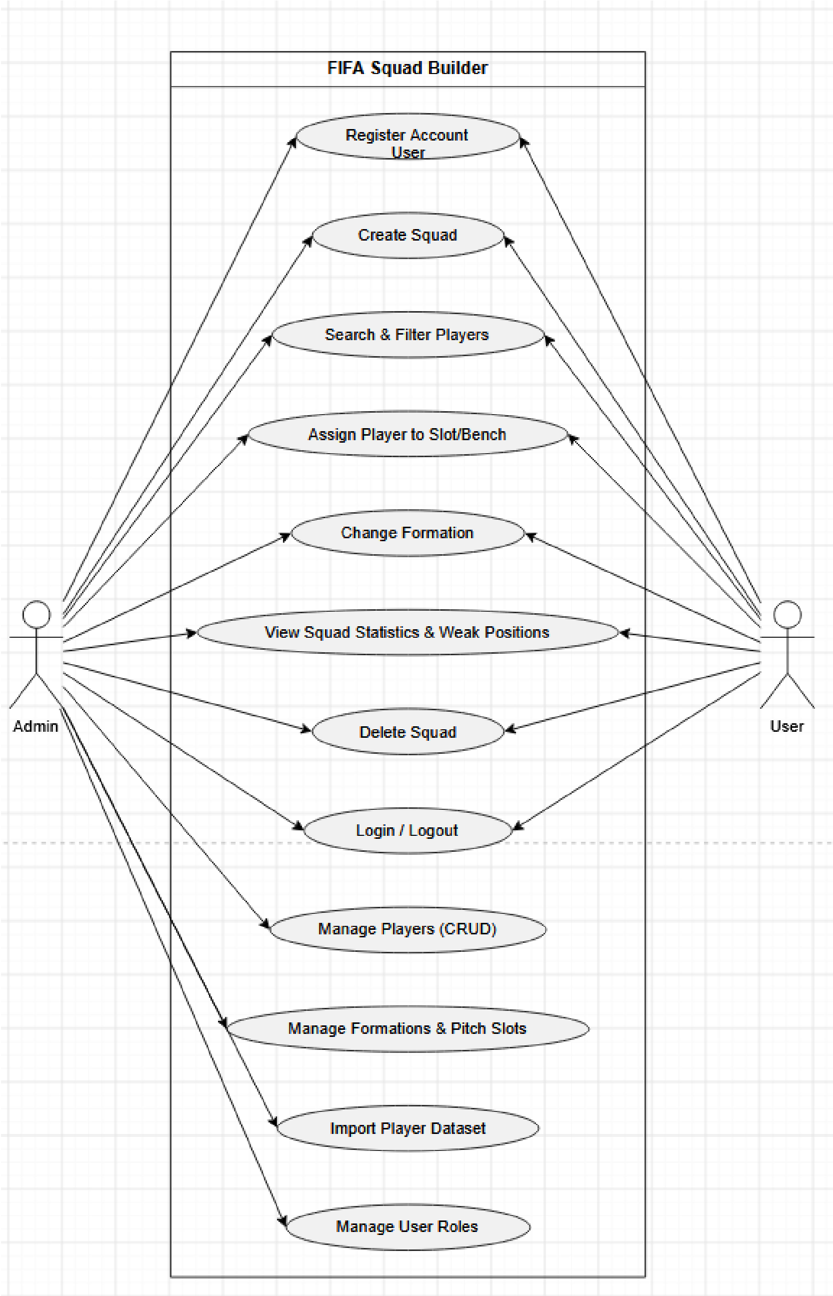


Figure 1: Use Case Diagram for FIFA Squad Builder

5.6. Real Project Screenshots

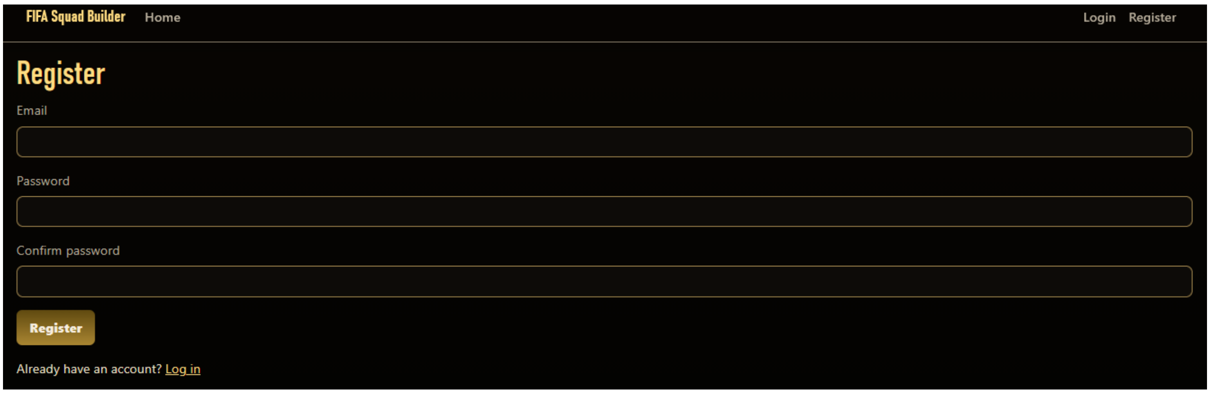


Figure 2: Register page

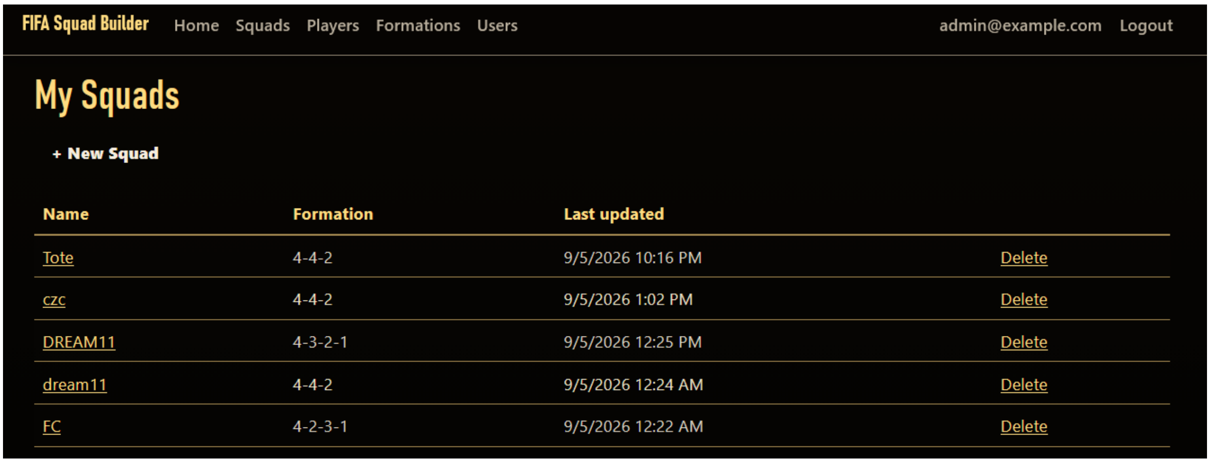


Figure 3: My Squads list, showing a user's saved squads

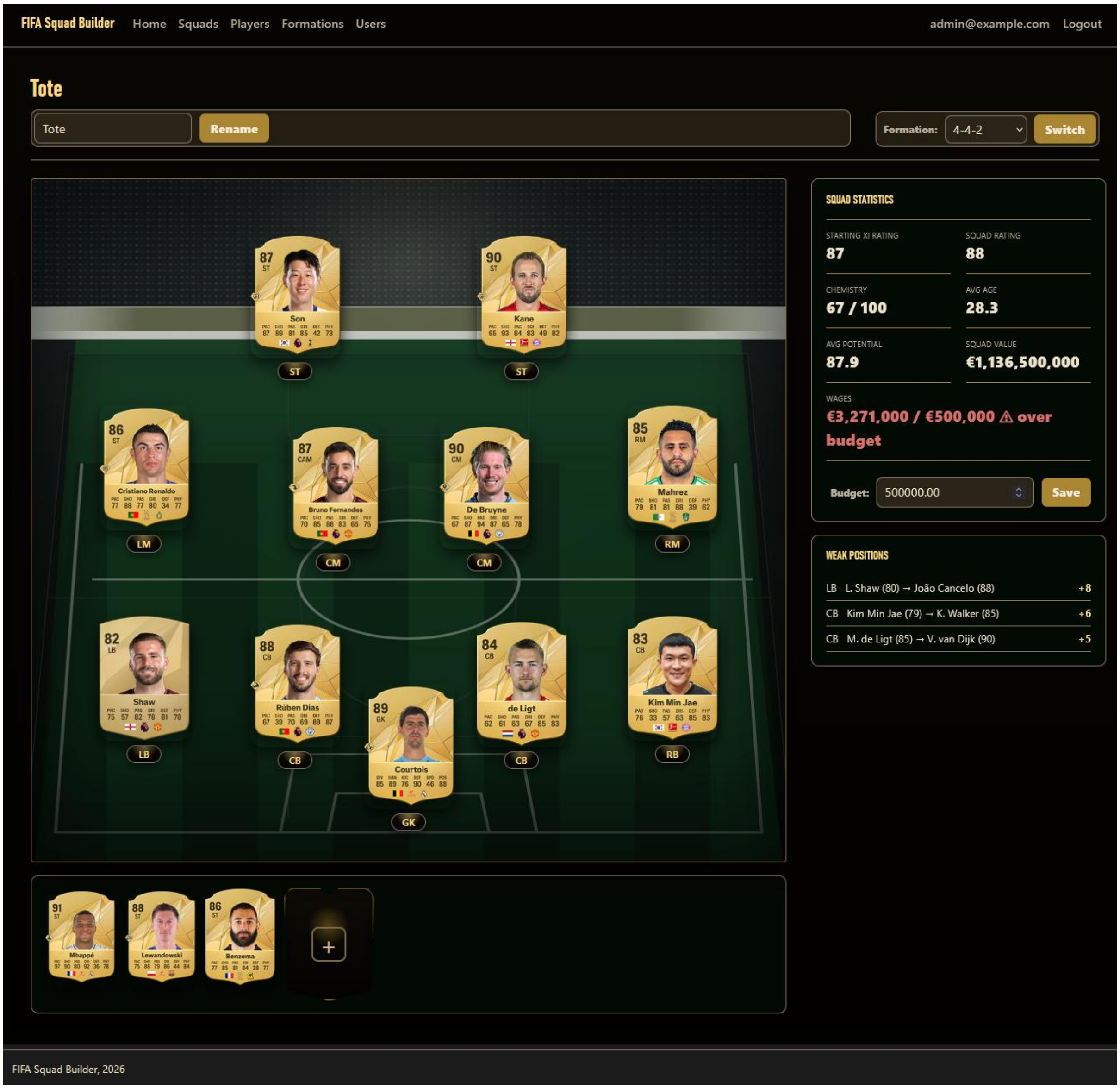
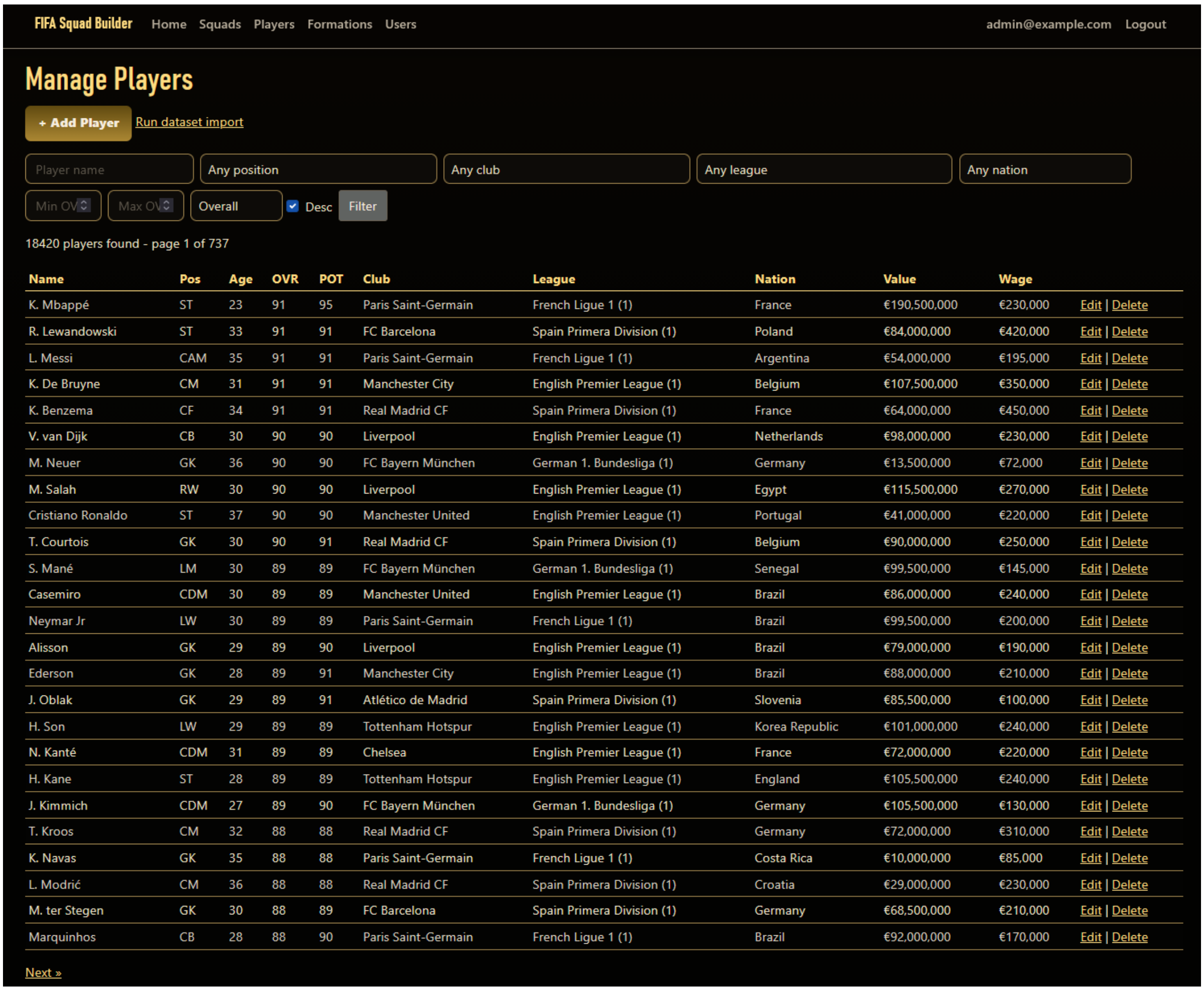
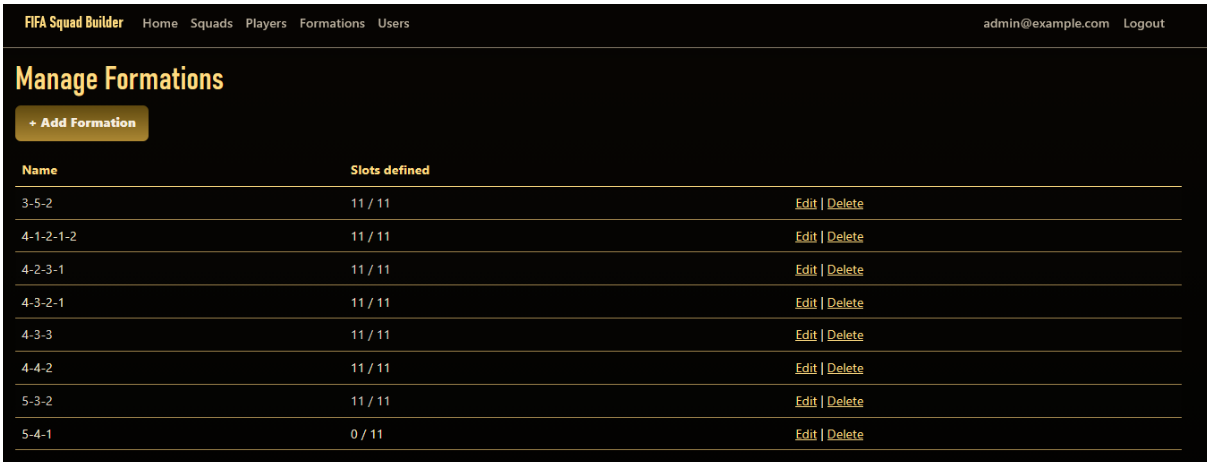


Figure 4: Squad pitch editor showing formation-positioned player cards and live statistics



Name	Pos	Age	OVR	POT	Club	League	Nation	Value	Wage	
K. Mbappé	ST	23	91	95	Paris Saint-Germain	French Ligue 1 (1)	France	€190,500,000	€230,000	Edit | Delete
R. Lewandowski	ST	33	91	91	FC Barcelona	Spain Primera Division (1)	Poland	€84,000,000	€420,000	Edit | Delete
L. Messi	CAM	35	91	91	Paris Saint-Germain	French Ligue 1 (1)	Argentina	€54,000,000	€195,000	Edit | Delete
K. De Bruyne	CM	31	91	91	Manchester City	English Premier League (1)	Belgium	€107,500,000	€350,000	Edit | Delete
K. Benzema	CF	34	91	91	Real Madrid CF	Spain Primera Division (1)	France	€64,000,000	€450,000	Edit | Delete
V. van Dijk	CB	30	90	90	Liverpool	English Premier League (1)	Netherlands	€98,000,000	€230,000	Edit | Delete
M. Neuer	GK	36	90	90	FC Bayern München	German 1. Bundesliga (1)	Germany	€13,500,000	€72,000	Edit | Delete
M. Salah	RW	30	90	90	Liverpool	English Premier League (1)	Egypt	€115,500,000	€270,000	Edit | Delete
Cristiano Ronaldo	ST	37	90	90	Manchester United	English Premier League (1)	Portugal	€41,000,000	€220,000	Edit | Delete
T. Courtois	GK	30	90	91	Real Madrid CF	Spain Primera Division (1)	Belgium	€90,000,000	€250,000	Edit | Delete
S. Mané	LM	30	89	89	FC Bayern München	German 1. Bundesliga (1)	Senegal	€99,500,000	€145,000	Edit | Delete
Casemiro	CDM	30	89	89	Manchester United	English Premier League (1)	Brazil	€86,000,000	€240,000	Edit | Delete
Neymar Jr	LW	30	89	89	Paris Saint-Germain	French Ligue 1 (1)	Brazil	€99,500,000	€200,000	Edit | Delete
Alisson	GK	29	89	90	Liverpool	English Premier League (1)	Brazil	€79,000,000	€190,000	Edit | Delete
Ederson	GK	28	89	91	Manchester City	English Premier League (1)	Brazil	€88,000,000	€210,000	Edit | Delete
J. Oblak	GK	29	89	91	Atlético de Madrid	Spain Primera Division (1)	Slovenia	€85,500,000	€100,000	Edit | Delete
H. Son	LW	29	89	89	Tottenham Hotspur	English Premier League (1)	Korea Republic	€101,000,000	€240,000	Edit | Delete
N. Kanté	CDM	31	89	89	Chelsea	English Premier League (1)	France	€72,000,000	€220,000	Edit | Delete
H. Kane	ST	28	89	89	Tottenham Hotspur	English Premier League (1)	England	€105,500,000	€240,000	Edit | Delete
J. Kimmich	CDM	27	89	90	FC Bayern München	German 1. Bundesliga (1)	Germany	€105,500,000	€130,000	Edit | Delete
T. Kroos	CM	32	88	88	Real Madrid CF	Spain Primera Division (1)	Germany	€72,000,000	€310,000	Edit | Delete
K. Navas	GK	35	88	88	Paris Saint-Germain	French Ligue 1 (1)	Costa Rica	€10,000,000	€85,000	Edit | Delete
L. Modrić	CM	36	88	88	Real Madrid CF	Spain Primera Division (1)	Croatia	€29,000,000	€230,000	Edit | Delete
M. ter Stegen	GK	30	88	89	FC Barcelona	Spain Primera Division (1)	Germany	€68,500,000	€210,000	Edit | Delete
Marquinhos	CB	28	88	90	Paris Saint-Germain	French Ligue 1 (1)	Brazil	€92,000,000	€170,000	Edit | Delete

Figure 5: Admin → Manage Players screen



Name	Slots defined	
3-5-2	11 / 11	Edit | Delete
4-1-2-1-2	11 / 11	Edit | Delete
4-2-3-1	11 / 11	Edit | Delete
4-3-2-1	11 / 11	Edit | Delete
4-3-3	11 / 11	Edit | Delete
4-4-2	11 / 11	Edit | Delete
5-3-2	11 / 11	Edit | Delete
5-4-1	0 / 11	Edit | Delete

Figure 6: Admin → Edit Formation screen

6. Implementation

This section describes each major module of the application along with a representative code sample and a short description of how it works. Screenshots for each module are marked as placeholders, to be inserted by the author.

6.1. Authentication & Authorization Module

Implements registration, login, logout, and role-based access using ASP.NET Core Identity's UserManager and SignInManager directly, rather than any custom password handling. Self-registration always grants the User role only - Admin is only ever assigned via a configuration-driven startup bootstrap or by an existing Admin, closing the obvious privilege-escalation hole of a signup form choosing its own role. Login uses Identity's built-in brute-force lockout and guards every post-login redirect against open-redirect attacks.

Figure 7: Login page

```
var user = await _userManager.FindByEmailAsync(model.Email);
if (user == null)
{
    ModelState.AddModelError(string.Empty, "Invalid login attempt.");
    return View(model);
}


// lockoutOnFailure: true enables Identity's built-in brute-force lockout.
var result = await _signInManager.PasswordSignInAsync(
    user, model.Password, model.RememberMe, lockoutOnFailure: true);
```

```
if (result.Succeeded)
{
    return RedirectToLocal(model.ReturnUrl);
}
```

6.2. Player Data Import Module

Reads the real FIFA 23 dataset (players_fifa23.csv, teams_fifa23.csv) and idempotently upserts Nations, Leagues, Clubs, and Players into MySQL. The players file has no League column at all - League is resolved by joining on club name against the teams file - and the literal string "Free agent" (not a blank field) represents a player with no club. Re-running the import updates existing rows, matched by the dataset's own player ID, rather than creating duplicates.

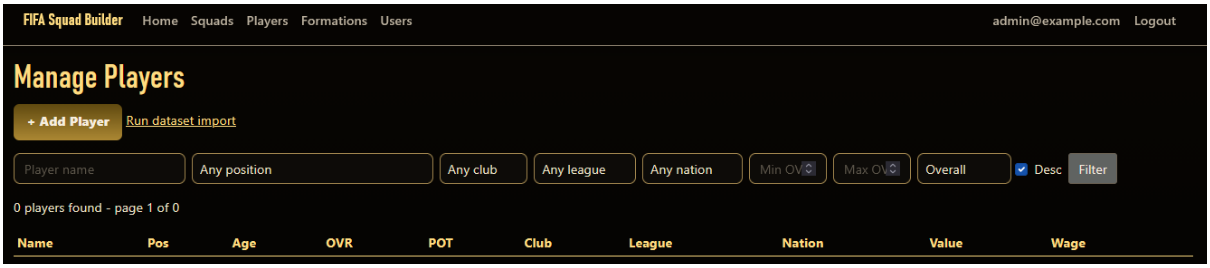


Figure 8: Admin → Import page

```
if (existingPlayers.TryGetValue(sourceId, out var existing))
{
    existing.Name = name;
    existing.Overall = overall;
    existing.ValueEUR = value;
    existing.WageEUR = wage;
    result.PlayersUpdated++;
}
else
{
    var player = new Models.Player { SourceId = sourceId, Name = name, /* ... */ };
    _db.Players.Add(player);
    result.PlayersCreated++;
```

```
}
```

6.3. Admin Player & Formation Management Module

Role-gated CRUD screens for the master Player and Formation data. Deleting a player that is still used in a saved squad is blocked at the database level (a foreign key with Restrict delete behavior) rather than silently allowed - the controller catches this and shows a plain-language message instead of letting a database error reach the user.

Figure 9: Admin → Edit Players screen

```
_db.Players.Remove(player);
try
{
    await _db.SaveChangesAsync();
```

```
    TempData["StatusMessage"] = $"Player '{player.Name}' deleted.";
}
catch (DbUpdateException)
{
    // FK is Restrict on purpose - a player still used in a saved squad
    // cannot be silently deleted out from under that user.
    TempData["StatusMessage"] =
        $"Cannot delete '{player.Name}' - it is still used in a saved squad.";
}
```

6.4. Squad Management Module

Lets a user create, rename, delete, and list their own squads. Every action is routed through a dedicated SquadService, which enforces ownership as part of the database query itself, not as a separate check performed after loading the data - a nonexistent squad id and someone else's squad id produce an identical "not found" result, so a manipulated URL cannot be used to probe for other users' data.

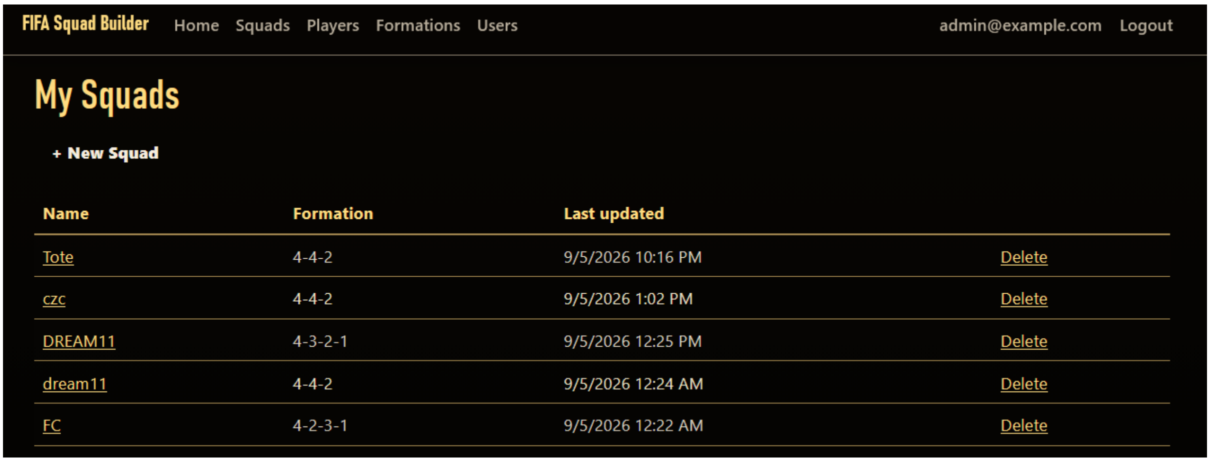


Figure 10: My Squads list

```
public async Task<SquadEditorData?> GetSquadForEditAsync(int squadId, string userId)
{
    // Ownership check is baked into the query itself - a squad belonging
    // to someone else simply doesn't match this query, so this method
    // can't be used to distinguish "not found" from "not yours".
```

```
var squad = await _db.Squads
    .Include(s => s.Formation).ThenInclude(f => f.Positions)
    .Include(s => s.SquadPlayers).ThenInclude(sp => sp.Player)
    .FirstOrDefaultAsync(s => s.Id == squadId && s.UserId == userId);
```

6.5. Player Management (Pitch Module)

Lets a user click a pitch slot or bench spot to search the full player database (server-side paginated, never loading the whole table to the browser) and assign a player. Assigning a player already in the squad moves their single record rather than creating a duplicate and assigning to an already-occupied slot automatically bumps the previous occupant to the bench (rejected if the bench is already full).

Figure 11: Squad pitch editor and player search modal

```
var occupant = squad.SquadPlayers
    .FirstOrDefault(sp => sp.FormationPositionId == formationPositionId.Value && sp.PlayerId != playerId);


if (occupant != null)
{
    if (benchCountExcludingMovingPlayer + 1 > BenchCapacity)
    {
        return SquadOperationResult.Fail(
            "Bench is full (7 max) - remove a substitute before replacing this position.");
    }
```

```
    occupant.FormationPositionId = null;

    occupant.IsStartingXI = false;

    occupant.SlotOrder = NextBenchSlotOrder(squad.SquadPlayers, excludePlayerId: playerId);

}
```

6.6. Chemistry Calculation Module

Implements this project's own, documented chemistry model - not a reproduction of EA's proprietary algorithm. Each starting player earns up to 3 points based on positional fit and shared club/league/nation with other starters; the squad's overall chemistry is the sum of every starter's points, scaled onto a 0-100 display range.

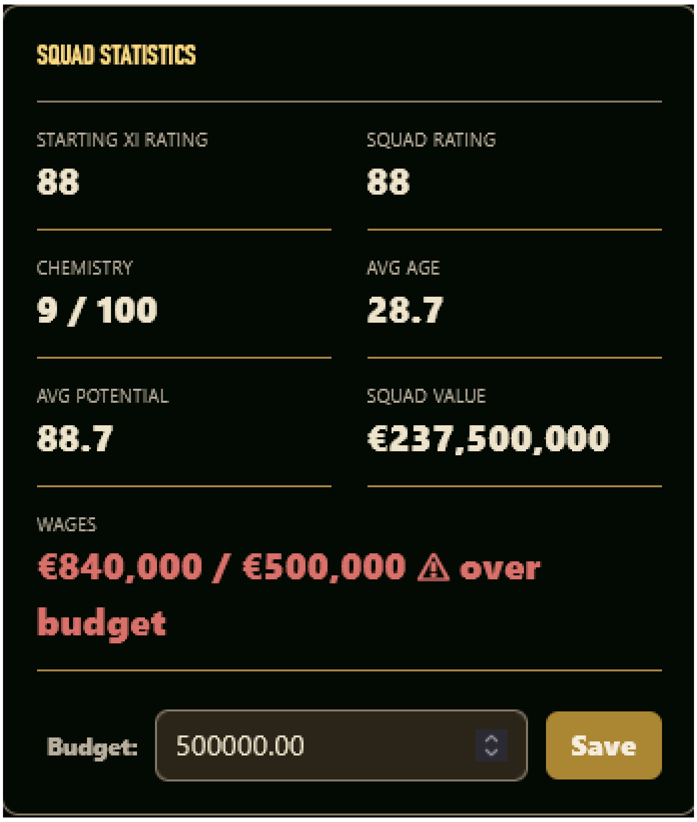


Figure 12: Chemistry score displayed in the squad statistics panel

```
if (player.AssignedPositionId == player.PlayerPositionId) points++;


if (player.ClubId.HasValue)
{
```

```
    var clubMates = startingXI.Count(p =>
        p.SquadPlayerId != player.SquadPlayerId && p.ClubId == player.ClubId);
    if (clubMates >= ClubLinkThreshold) points++;   // >= 2 other clubmates
}

points = Math.Min(points, MaxPointsPerPlayer);   // capped at 3 per player
```

6.7. Squad Statistics & Weak-Position Module

Computes Starting XI rating, overall squad rating, average age/potential, squad value, and wage-budget status (with an over-budget warning). A separate weak-position service compares each starter against the best-rated player at their exact position who is not already anywhere in the squad and reports it only if the gap is significant.

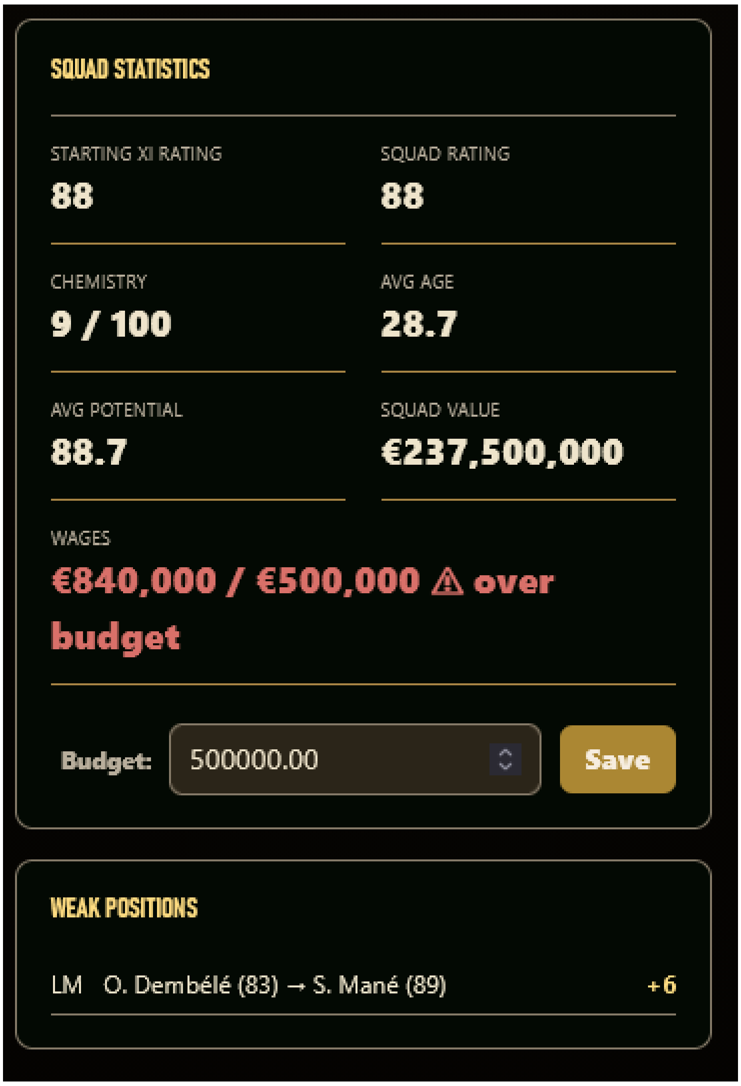


Figure 13: Weak-position recommendations panel

```
var bestAvailable = await _db.Players
    .Where(p => p.PositionId == requiredPositionId && !playerIdsInSquad.Contains(p.Id))
    .OrderByDescending(p => p.Overall)
    .Select(p => new { p.Name, p.Overall })
    .FirstOrDefaultAsync();


if (bestAvailable == null) continue; // no alternative exists at this position


var difference = bestAvailable.Overall - sp.Player.Overall;
if (difference >= SignificanceThreshold)   // >= 5 points
{
```

```
    results.Add(new WeakPositionEntry { /* ... */ Difference = difference });
}
```

6.8. Admin User Management Module

Lets an Admin promote or demote any other account's Admin status. An admin is explicitly blocked from changing their own role from this screen, which prevents an admin from accidentally locking themselves out of the system.

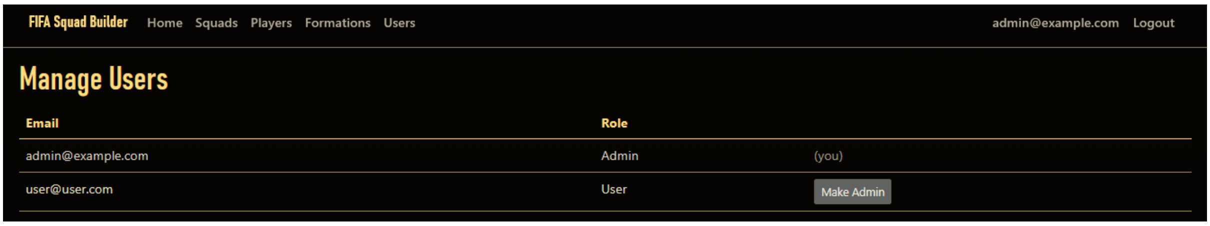


Figure 14: Admin → Manage Users screen

```
var currentUserId = _userManager.GetUserId(User);
if (id == currentUserId)
{
    // Prevents an admin from locking themselves out of their own access.
    TempData["StatusMessage"] = "You cannot change your own admin status.";
    return RedirectToAction(nameof(Index));
}


var isAdmin = await _userManager.IsInRoleAsync(user, AdminRoleName);
if (isAdmin) await _userManager.RemoveFromRoleAsync(user, AdminRoleName);
else await _userManager.AddToRoleAsync(user, AdminRoleName);
```

7. Result Analysis

The application was tested by running it against a real MySQL database, registering an account, creating a squad, assigning real imported players to a formation, and exercising the admin screens. The table below summarizes the outcome of testing for each major module.

Module	Expected Result	Status
Authentication	User can register, log in, and log out; roles are enforced on restricted pages	Working
Player Data Import	Real FIFA 23 dataset rows are imported/updated without duplication	Working
Admin Player & Formation Management	Admin can create/edit/delete players and formations; deletion is blocked if still in use	Working
Squad Management	User can create, rename, and delete squads; cannot access another user's squad	Working
Player Assignment	Player can be assigned to a pitch slot or bench; duplicates and slot conflicts are prevented	Working
Chemistry Calculation	Chemistry score reflects positional fit and club/league/nation links of the starting XI	Working
Squad Statistics & Weak Position	Rating, age, potential, value, and wage-budget figures update correctly; weak positions are identified	Working
Admin User Management	Admin can promote/demote other accounts; cannot change their own role	Working

During testing, two real issues were identified and resolved. First, the formation-selection dropdown on the squad pitch page did not reflect the squad's actual current formation - it defaulted to whichever formation name sorted first alphabetically, which was fixed by explicitly marking the correct option as selected based on the squad's stored formation. Second, the goalkeeper's card was visually clipped at the bottom edge of the pitch, caused by the pitch container's overflow being hidden while the goalkeeper's slot coordinate sat right at that edge; this was fixed by separating the pitch's background layer (which needs clipped, rounded corners) from the layer that holds the player cards (which must not be clipped). These issues and fixes reflect the iterative testing approach used throughout the project.

8. Non-Functional Requirements

- Performance: player search and listing pages use server-side pagination, so the browser never loads the full ~18,500-row player table at once.
- Security: every squad operation enforces ownership server-side as part of the database query; passwords are handled entirely by ASP.NET Core Identity, never custom code.
- Usability: the squad pitch, statistics panel, and weak-position list are designed to be understood at a glance without a manual.
- Portability: the application runs on any machine with the .NET 8 runtime and a reachable MySQL server; no other external dependency is required.
- Maintainability: business logic lives in dedicated services (Squad, Chemistry, Statistics, Weak-Position, Import, Search) rather than in controllers or views, keeping each concern isolated and independently testable.

9. Conclusion

This project successfully delivers a complete, working full-stack web application built with ASP.NET Core MVC and C#, covering secure role-based authentication, a real-data-backed relational database, server-enforced squad-ownership and business rules, and an original, documented chemistry/rating/weak-position system. The project demonstrates practical use of layered architecture (controllers, services, view models), Entity Framework Core relationship and constraint design, and automated testing using xUnit with the EF Core InMemory provider. The realistic, well-scoped design allowed the project to be completed, tested, and refined within the project timeline.

10. Recommendations

- Add drag-and-drop player assignment as an alternative to the current click-to-search flow.
- Populate real ISO nation codes from a proper lookup table instead of leaving them unset.
- Extend the admin screens' visual styling to fully match the squad-builder page's polish.

11. Future Scope

Future versions of this application could include a full drag-and-drop squad editor, real-time multi-device sync of a squad in progress, social features such as sharing or comparing squads between users, additional statistical breakdowns (e.g. per-position chemistry heatmaps), and support for additional or custom datasets beyond the current FIFA 23 dataset.